UNIVERSIDAD CUAUHTÉMOC

Ingeniería en Sistemas Computacionales

PROYECTO FINAL

Desarrollo de una Aplicación Básica en Flutter

Videojuegos Disada

Integrantes:

Daniel Martínez Sánchez

José Salvador Montes Luna

Diego Eduardo Hurtado Hernández

Materia: Desarrollo Móvil con Flutter

Abril 2026

1. Resumen del Proyecto

1.1 Descripción Breve

Game Vault — Videojuegos Disada es una aplicación móvil desarrollada en Flutter que simula una tienda en línea de videojuegos. Permite explorar un catálogo extenso organizado por géneros, seleccionar títulos para comprar, gestionar favoritos y completar un proceso de compra con selección de método de pago.

1.2 Público Objetivo

- Gamers y aficionados a los videojuegos entre 15 y 35 años.
- Compradores casuales que deseen explorar el catálogo por géneros antes de decidir su compra.
- Usuarios que aprecian interfaces gaming con dark theme.

1.3 Principales Funcionalidades

- Pantalla Home con accesos directos y estadísticas en tiempo real.
- Tienda de Videojuegos: catálogo con más de 50 juegos por género, precios en MXN y calificaciones.
- Proceso de compra con formulario del cliente y selección múltiple de juegos.
- Método de pago: Efectivo o Tarjeta (débito/crédito/transferencia).
- Pantalla de confirmación exitosa con número de pedido.
- Explorador de juegos con buscador en tiempo real.
- Mis Favoritos persistidos localmente con Hive.

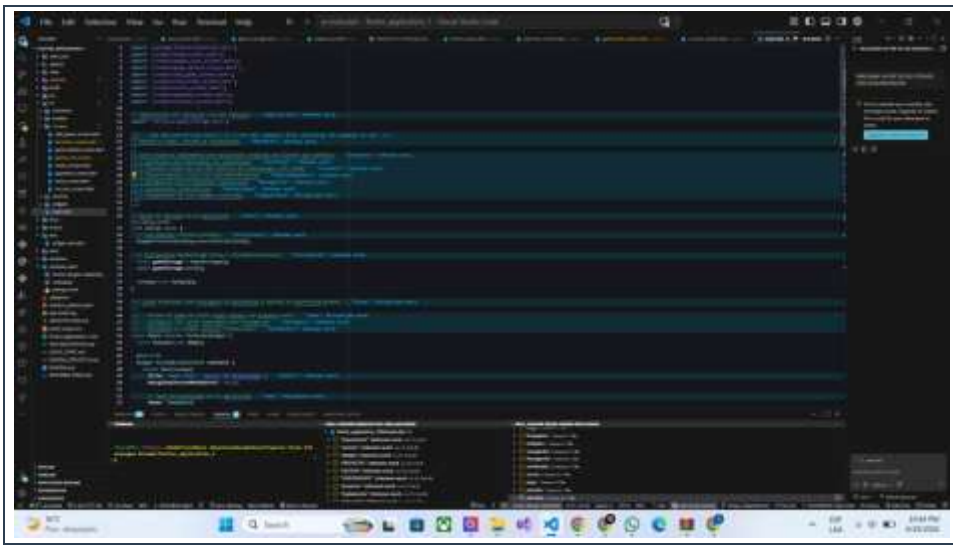
2. Arquitectura de la Aplicación

2.1 Estructura de Carpetas y Archivos

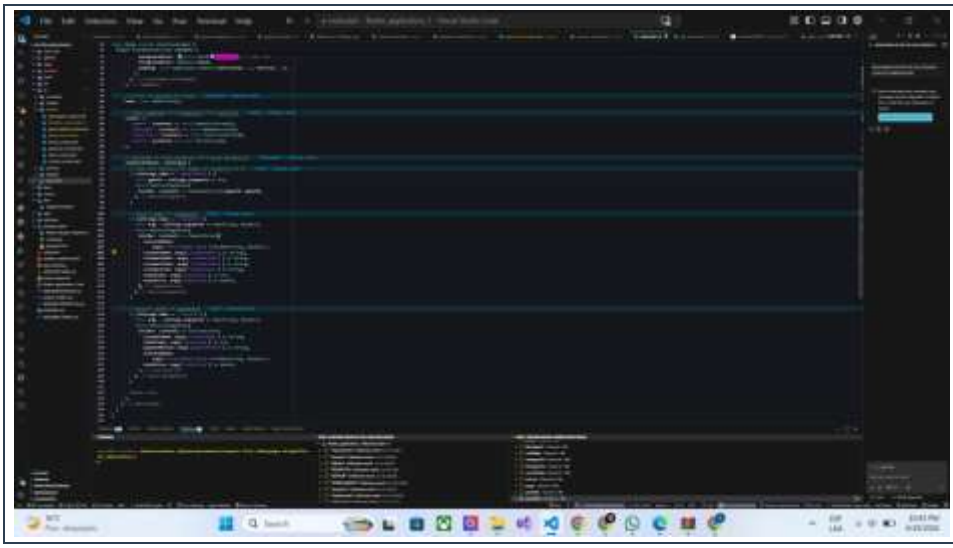
Carpeta / Archivo	Descripción
lib/main.dart	Punto de entrada. Inicializa Hive, SharedPreferences y rutas nombradas.
lib/screens/home_screen.dart	Pantalla principal con bienvenida, accesos directos y estadísticas.
lib/screens/add_game_screen.dart	Tienda con formulario de cliente y catálogo por géneros.
lib/screens/payment_screen.dart	Pantalla de método de pago con resumen de compra.
lib/screens/success_screen.dart	Pantalla de confirmación de compra exitosa.
lib/screens/games_list_screen.dart	Explorador del catálogo completo con búsqueda y paginación.
lib/screens/favorites_screen.dart	Lista de juegos favoritos con ordenamiento (nombre, calificación, año).
lib/models/game.dart	Modelo Game con adaptador Hive (HiveType, HiveField).
lib/services/game_storage.dart	GameStorage: capa de acceso a Hive y SharedPreferences.
lib/services/api_service.dart	ApiService: peticiones HTTP a la API RAWG.
lib/widgets/game_card.dart	Widget reutilizable GameCard para tarjetas de juego.
lib/constants/app_constants.dart	Constantes globales: colores, tamaños, textos.

2.2 Código Fuente — main.dart (Configuración e Importaciones)

El archivo main.dart es el punto de entrada de la aplicación. Configura las importaciones de todas las pantallas y servicios, inicializa Hive y SharedPreferences mediante GameStorage, define el ThemeData con el tema oscuro gaming y registra todas las rutas nombradas.



Código 1 — main.dart: importaciones, void main() e inicialización de GameStorage



Código 2 — main.dart: ThemeData, rutas nombradas (routes) y onGenerateRoute con paso de parámetros

2.3 Flujo de Navegación

La aplicación usa Named Routes configuradas en main.dart con onGenerateRoute para pasar datos complejos (lista de juegos seleccionados, datos del cliente, total) entre pantallas:

Home (/) → Tienda (/add-game) → Pago (/payment) → Éxito (/success) → Home

Home (/) → Lista de Juegos (/games) → [back] Home

Home (/) → Mis Favoritos (/favorites) → [back] Home

Tienda (/add-game) → Detalle de Juego (/game-detail) → [back] Tienda

3. Diseño de la Interfaz de Usuario (UI)

3.1 Lista de Widgets Clave

Widget	Pantalla	Propósito
ListView	Home, Tienda, Favoritos	Scrolling vertical de listas y catálogo por género.
GridView (horizontal)	Tienda, Lista de Juegos	Cuadrícula deslizable de juegos con 2 columnas.
TextField + Controller	Tienda, Explorador	Formulario del cliente y buscador en tiempo real.
StatefulWidget + setState	Todas	Actualiza la UI al seleccionar juegos o favoritos.
AnimatedContainer	Tienda	Animación de selección en tarjetas de juego.
RadioListTile	Pago	Selección excluyente de método de pago.
PopupMenuButton	Favoritos	Menú desplegable para ordenar por nombre, calificación o año.
ScaleTransition	Compra Exitosa	Animación de escala en ícono de confirmación.
Container + BoxDecoration	Todas	Tarjetas con bordes de colores y esquinas redondeadas.

3.2 Capturas de Pantalla — Pantallas de la Aplicación

Pantalla 1 — Home Screen (Inicio)

Banner de bienvenida, tres accesos directos (Comprar, Lista de Juegos, Mis Favoritos) y cuatro tarjetas de estadísticas con bordes de colores.

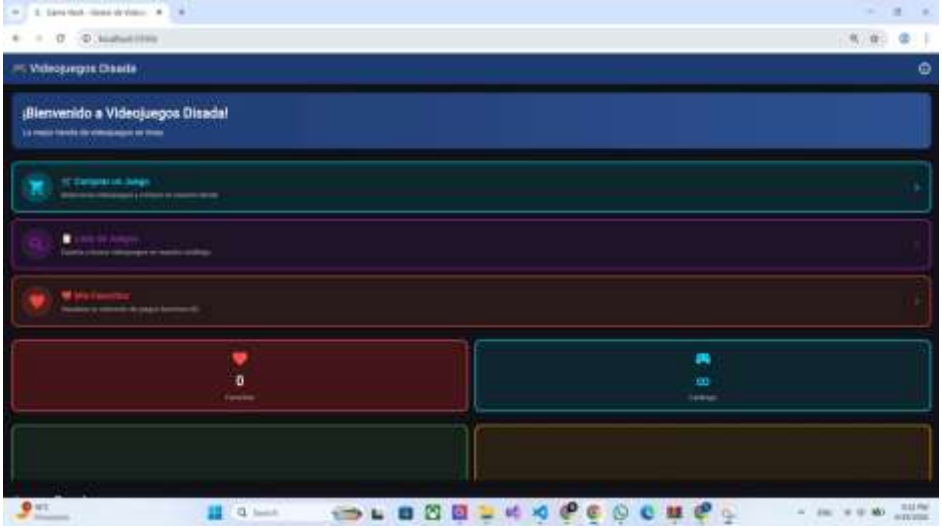


Figura 1 — Pantalla de Inicio (Home Screen)

Pantalla 2 — Tienda de Videojuegos

Formulario de datos del cliente en la parte superior, catálogo de juegos agrupado por géneros con scroll horizontal. Los juegos seleccionados se resaltan con borde cyan.

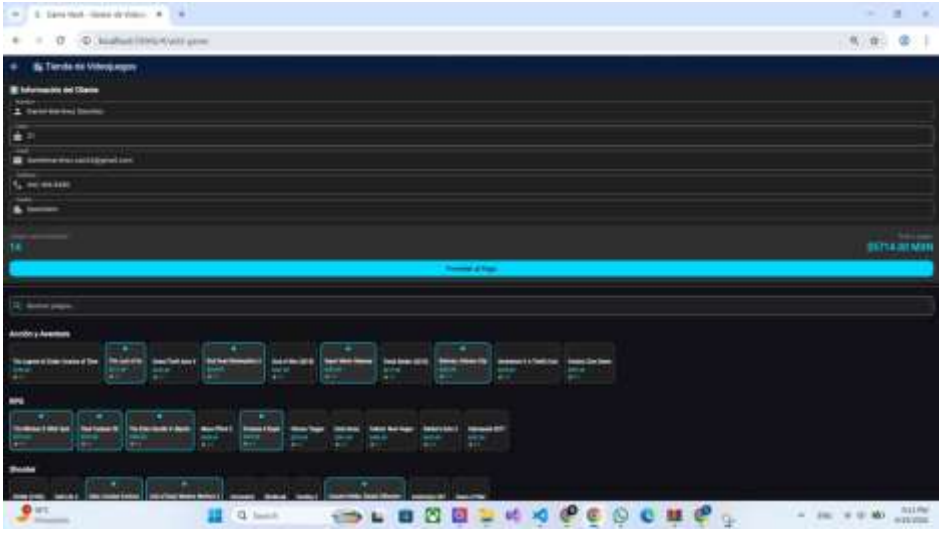


Figura 2 — Tienda de Videojuegos con catálogo por géneros

Pantalla 3 — Método de Pago

Resumen detallado de todos los juegos seleccionados con precios individuales, total a pagar y datos del cliente. Permite elegir entre Pago en Efectivo o Pago con Tarjeta.

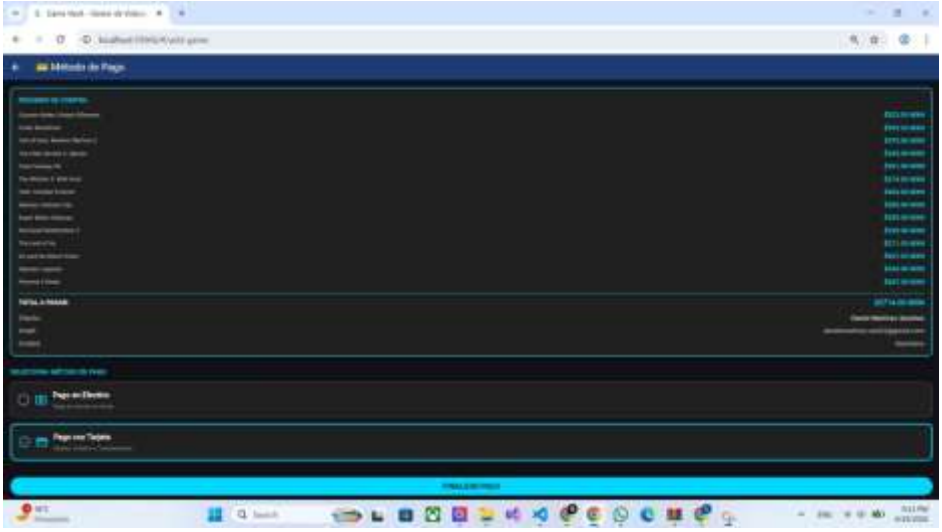


Figura 3 — Resumen de compra y método de pago

Pantalla 4 — Compra Exitosa

Confirmación con ícono verde animado (ScaleTransition). Muestra nombre del cliente, número de juegos, método de pago, monto y número de confirmación aleatorio.

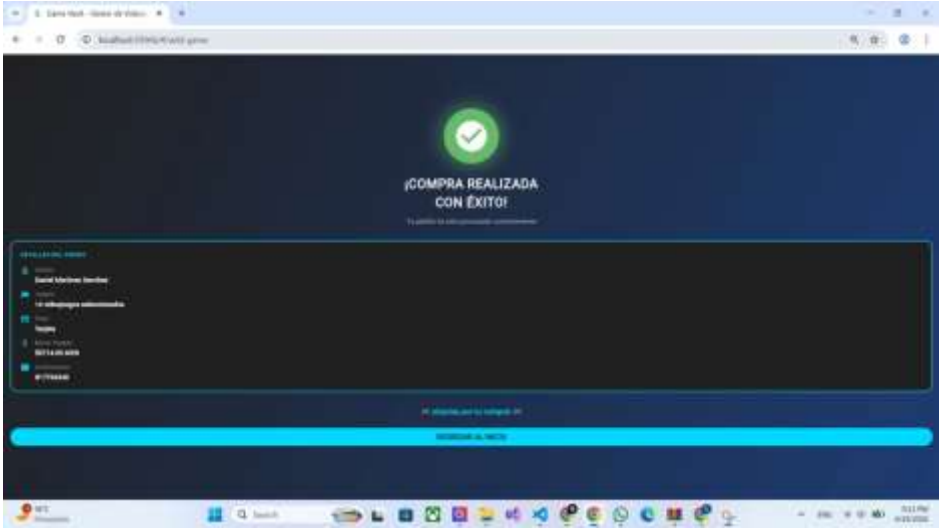


Figura 4 — Pantalla de compra exitosa

Pantalla 5 — Explorador de Juegos

Catálogo completo con buscador en tiempo real. Paginación infinita con botón 'Cargar más'. Muestra estado vacío ilustrativo cuando no hay resultados.

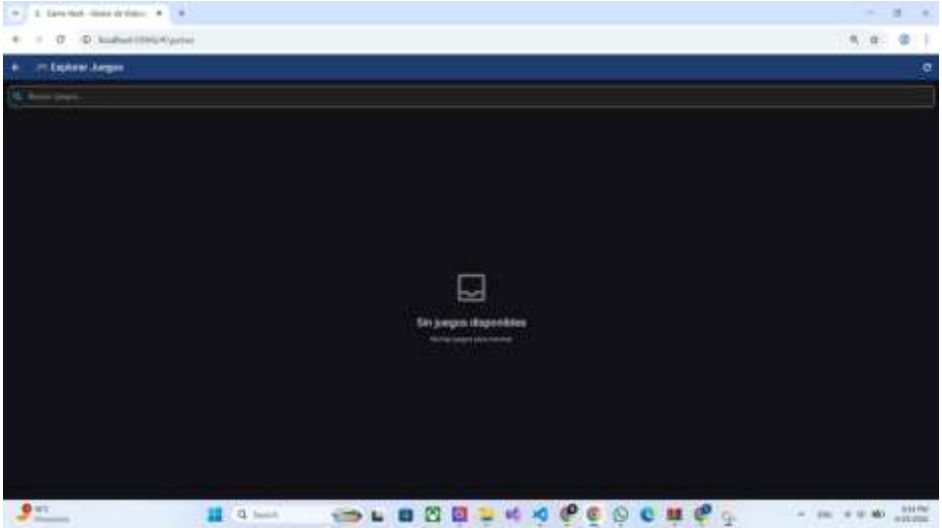


Figura 5 — Explorador de juegos con buscador

Pantalla 6 — Mis Favoritos

Lista de juegos guardados como favoritos persistidos con Hive. Ordenamiento por nombre, calificación o año mediante PopupMenuButton. Estado vacío con ilustración.

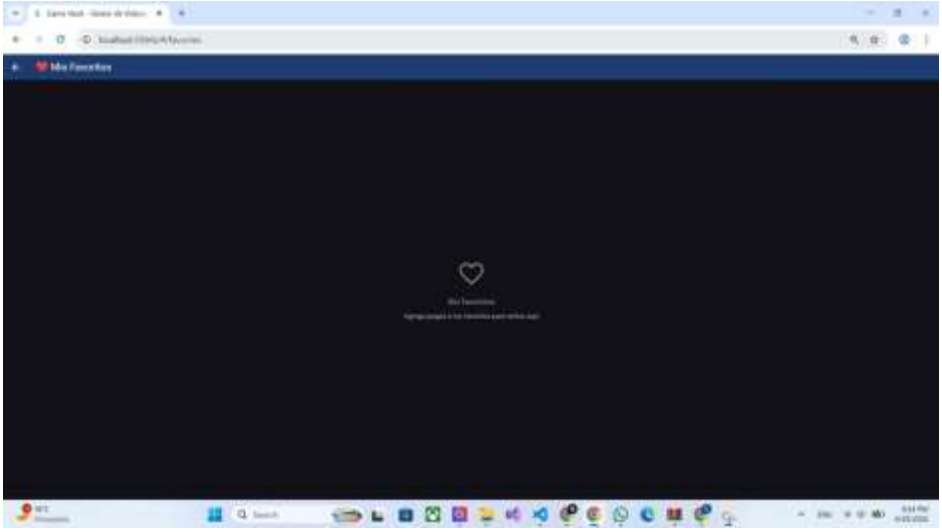
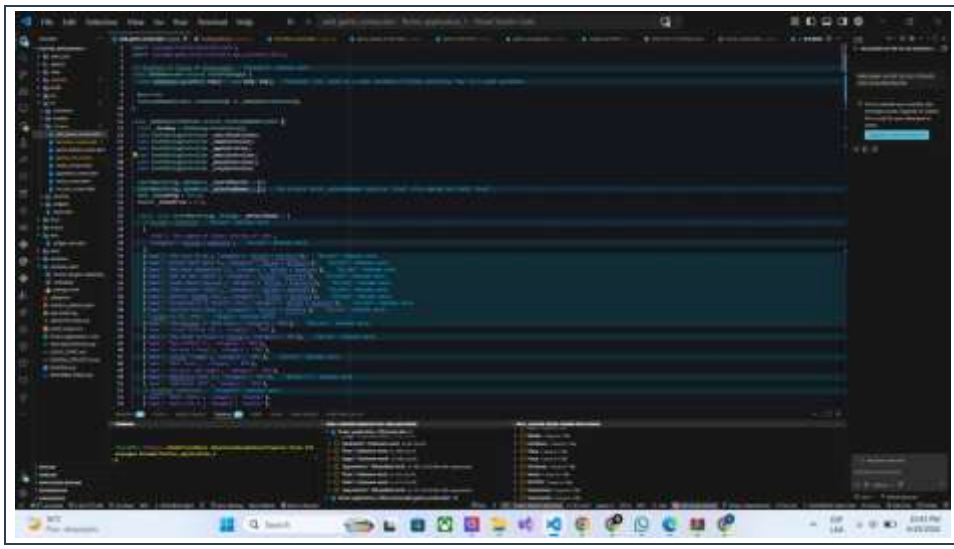


Figura 6 — Pantalla de Mis Favoritos

3.3 Código Fuente — Pantallas Principales

add_game_screen.dart — Tienda de Videojuegos

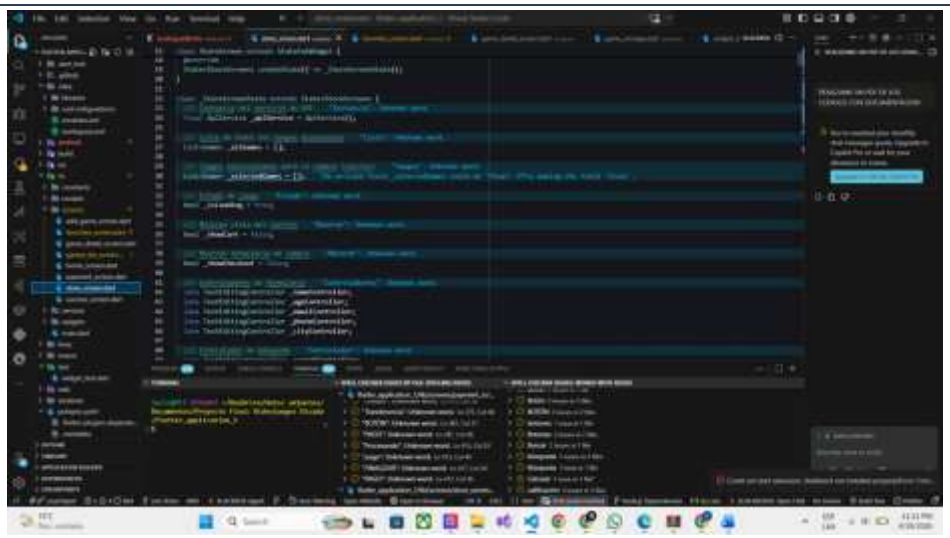
Pantalla principal de compra. Implementa StatefulWidget con GlobalKey<FormState>, múltiples TextEditingController para el formulario del cliente, y la lista estática de juegos por categoría (_defaultGames) con más de 50 títulos distribuidos en géneros (Acción y Aventura, RPG, Shooter, Plataforma, Puzzle, etc.).



Código 3 — add_game_screen.dart: declaración de clase, controladores y catálogo de juegos

store_screen.dart — Tienda con API (Parte 1)

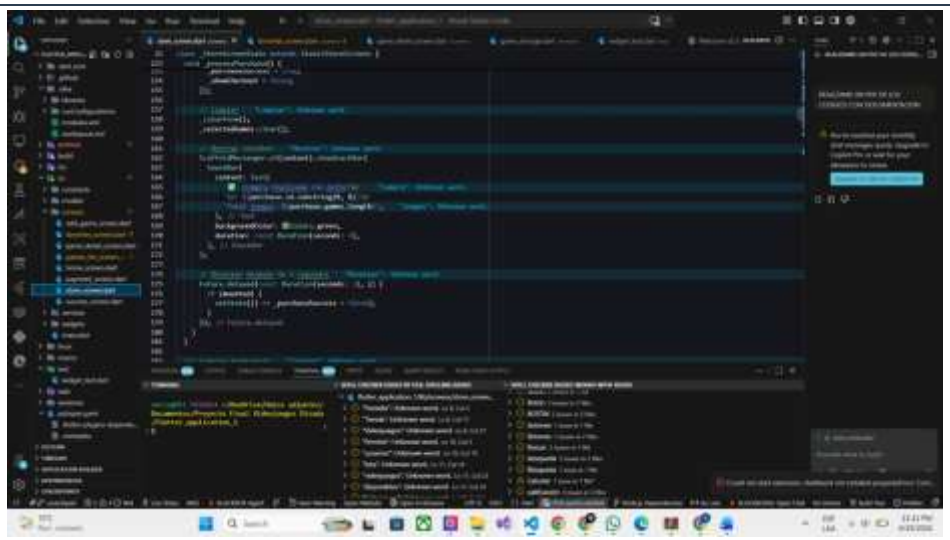
Versión alternativa de la tienda que consume la API en tiempo real. Declara ApiService para cargar juegos dinámicamente, la lista _allGames con todos los juegos disponibles, _selectedGames como carrito de compra, los estados _isLoading / _showCart / _showCheckout, y los mismos TextEditingController del formulario de cliente.



Código 8 — store_screen.dart: ApiService, listas de juegos, estados booleanos y controladores de formulario

store_screen.dart — Tienda con API (Parte 2 — processPurchase)

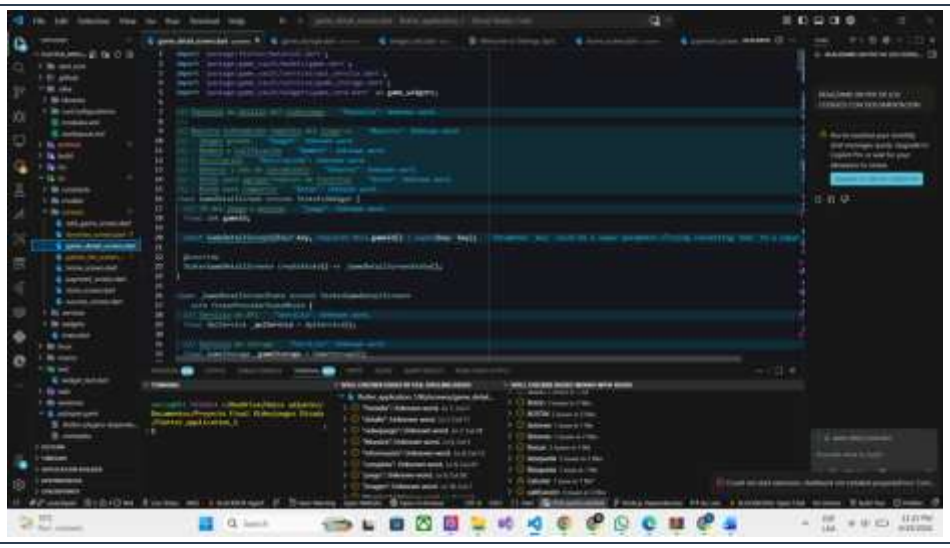
Función _processPurchase() que finaliza la compra: activa _purchaseSuccess, limpia el formulario y el carrito, muestra un SnackBar verde con el ID de compra y el número de juegos adquiridos, y usa Future.delayed para restablecer el estado después de 2 segundos.



Código 9 — store_screen.dart: _processPurchase(), SnackBar de confirmación y reset con Future.delayed

game_detail_screen.dart — Detalle del Videojuego

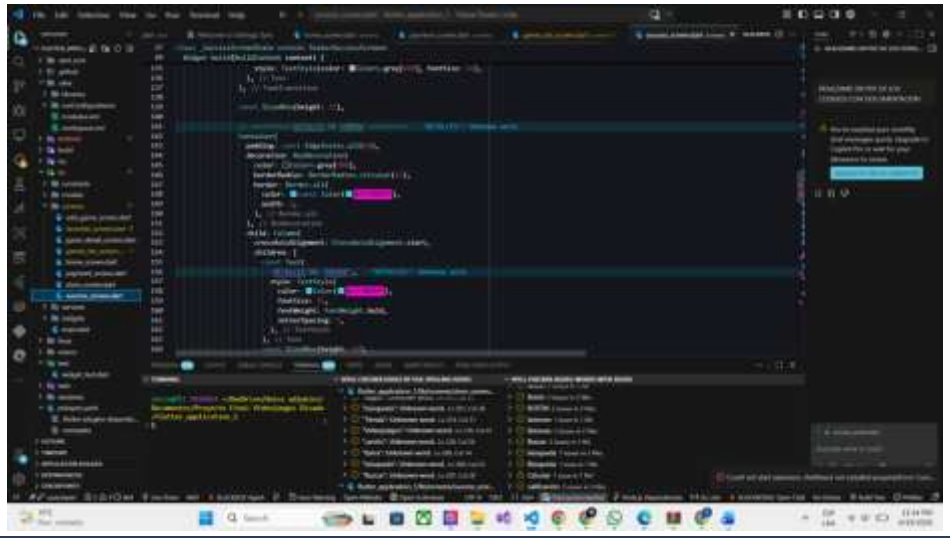
Pantalla de detalle del juego que recibe el gameId como parámetro requerido. Implementa StatefulWidget con TickerProviderStateMixin para animaciones. Utiliza ApiService para cargar la información completa del juego y GameStorage para gestionar el estado de favorito. Muestra imagen grande, nombre, descripción, géneros, año de lanzamiento y calificación.



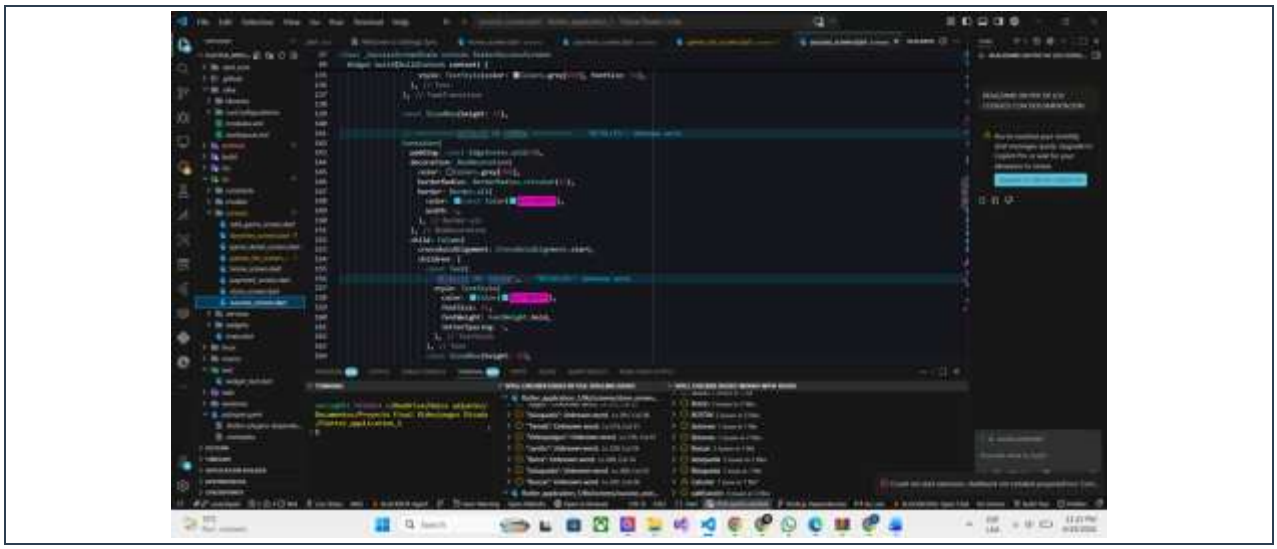
Código 11 — game_detail_screen.dart: importaciones, clase StatefulWidget con gameId, ApiService y GameStorage

success_screen.dart — Pantalla de Compra Exitosa (UI)

La sección de DETALLES DE COMPRA usa un Container con BoxDecoration (borderRadius circular, borde de color 0xFF00095FF cyan), Column con crossAxisAlignment.start y el texto 'DETALLES DEL PEDIDO' con letterSpacing, FontWeight.bold y fontSize 12 en cyan. Combina ScaleTransition y FadeTransition para la entrada animada del ícono y el texto de confirmación.



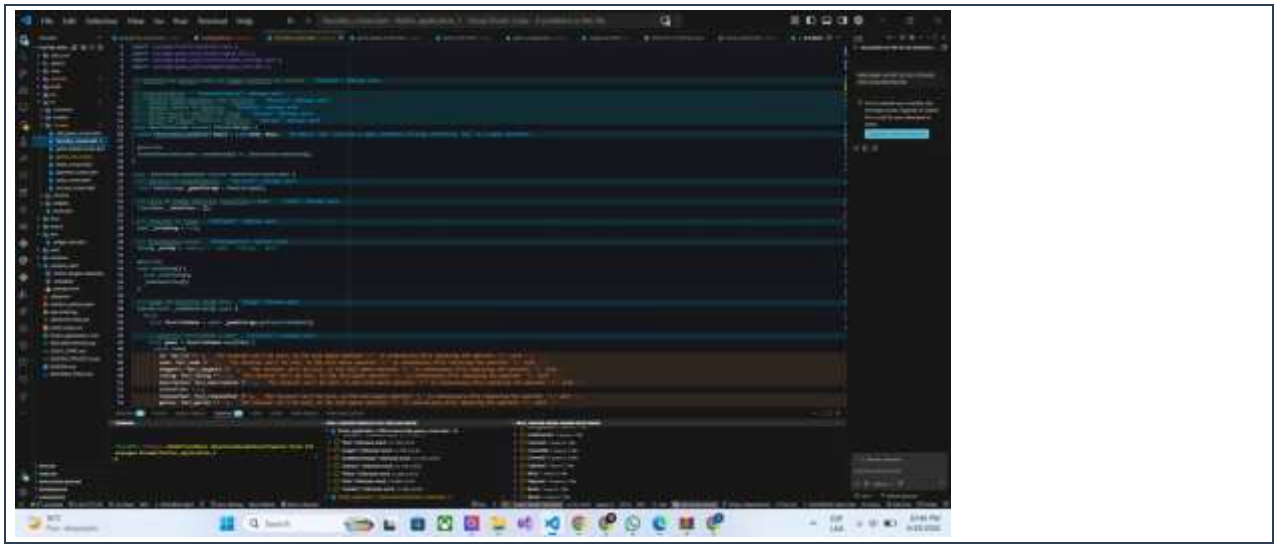
Código 10 — success_screen.dart: Container con BoxDecoration, sección DETALLES DEL PEDIDO con estilos



Código 12 — success_screen.dart: continuación del widget build() con detalles de pedido y estilos (vista idéntica, scroll diferente)

favorites_screen.dart — Mis Favoritos (Parte 1)

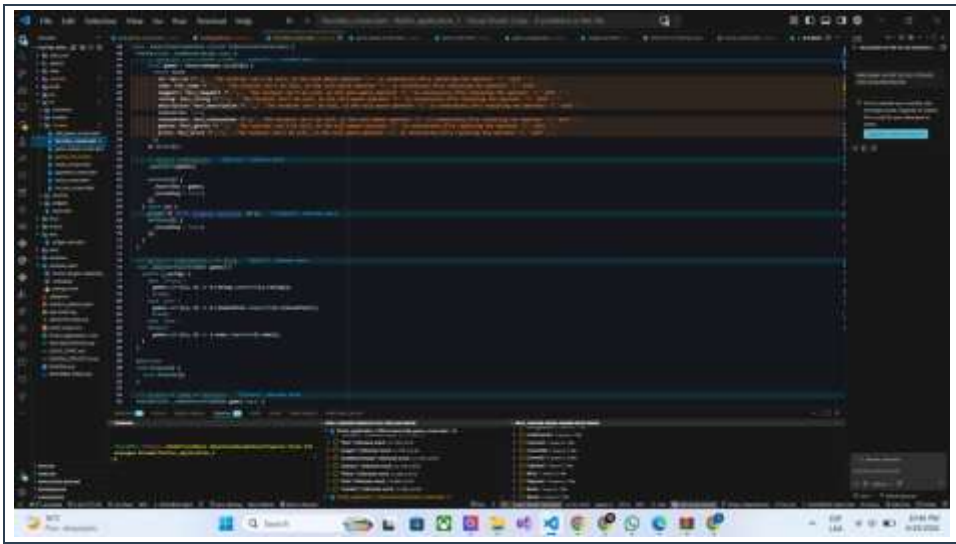
Carga los favoritos desde Hive mediante GameStorage.getFavoriteGames(). Convierte los registros HiveObject en objetos Game con null-safety (??) y aplica el ordenamiento seleccionado. Gestiona estados de carga con _isLoading.



Código 4 — favorites_screen.dart: loadFavorites(), conversión de HiveObject a Game y setState

favorites_screen.dart — Mis Favoritos (Parte 2 — Ordenamiento)

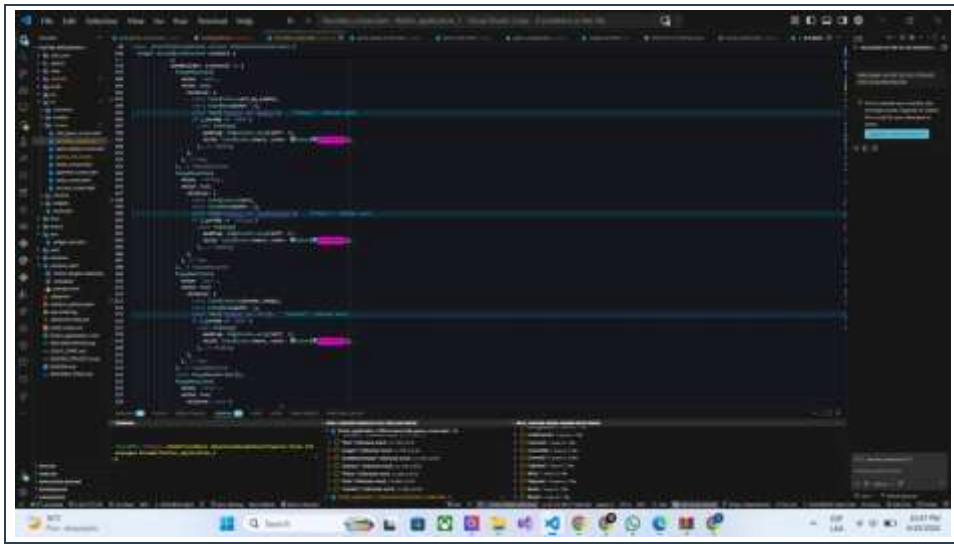
Función _applySort() con switch-case para ordenar la lista de favoritos por calificación (rating), año de lanzamiento (releaseYear) o nombre alfabético (name). Se llama automáticamente después de cargar o modificar la lista.



Código 5 — favorites_screen.dart: _applySort(), dispose() y _removeFavorite()

favorites_screen.dart — Mis Favoritos (Parte 3 — UI de Ordenamiento)

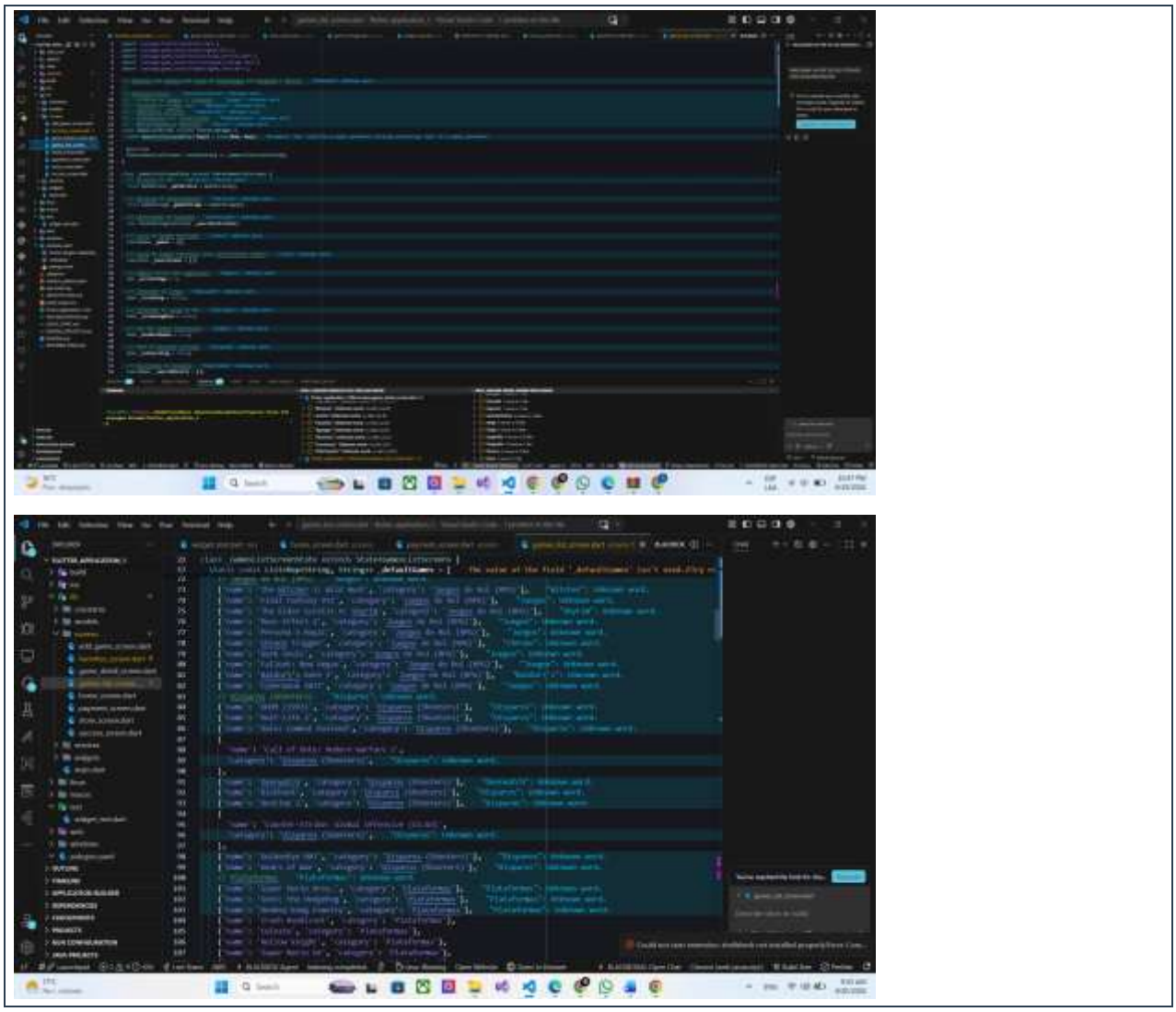
PopupMenuButton con tres opciones de ordenamiento (Nombre, Calificación, Año) y un separador para la opción 'Limpiar'. Cada opción muestra un ícono check en color cyan cuando está seleccionada, usando condicionales en el árbol de widgets.



Código 6 — favorites_screen.dart: PopupMenuButton con opciones de ordenamiento y checkmarks

games_list_screen.dart — Explorador de Juegos

Pantalla de exploración del catálogo completo. Implementa ApiService para cargar juegos desde RAWG, GameStorage para acceso local, paginación con _currentPage y _hasMoreGames, búsqueda en tiempo real con _isSearching/_searchResults, e indicadores de carga diferenciados (_isLoading / _isLoadingMore).



Código 7 — games_list_screen.dart: declaración de variables, servicios y controladores de estado

4. Consumo de API

4.1 API Utilizada

Campo	Detalle
Nombre	RAWG Video Games Database API
URL Base	https://api.rawg.io/api
Endpoint Principal	https://api.rawg.io/api/games?key={API_KEY}&page_size=20&page={page}
Método HTTP	GET
Formato de Respuesta	JSON con: id, name, genres[], rating, background_image, released.
Librería Flutter	http (^1.1.0) — paquete oficial para peticiones HTTP en Dart.
Clase responsable	lib/services/api_service.dart — ApiService

4.2 Descripción del Funcionamiento

Al iniciar la pantalla GamesListScreen, ApiService realiza una petición GET paginada a RAWG (page=1, page_size=20). La respuesta JSON se deserializa en objetos Game. Al llegar al final del scroll o presionar 'Cargar más', se incrementa _currentPage y se solicita la siguiente página.

Los datos descargados se almacenan en caché dentro de un box Hive (gamesBox) para acceso offline. En caso de error de red se muestra un widget de error con opción de reintentar.

4.3 Manejo de Errores y Estados de Carga

- Estado de carga inicial: CircularProgressIndicator centrado (_isLoading = true).
- Carga de más páginas: indicador en la parte inferior (_isLoadingMore = true).
- Error de red: bloque try/catch con mensaje amigable y botón de reintentar.
- Sin resultados: widget 'Sin juegos disponibles' con ícono ilustrativo.
- Timeout: Duration de 15 segundos máximo por petición HTTP.

5. Persistencia de Datos y Animaciones

5.1 Capa de Acceso — GameStorage

La clase GameStorage (lib/services/game_storage.dart) centraliza todo el acceso a Hive y SharedPreferences. Se inicializa en main() mediante await gameStorage.init() antes de runApp(), garantizando que la base de datos esté lista al arrancar la app.

5.2 Persistencia con Hive

Box (Tabla Hive)	Datos almacenados	Propósito
favoritesBox	Lista de objetos Game (HiveObject)	Persiste favoritos del usuario entre sesiones.
purchasesBox	Historial de objetos Pedido con juegos, total y método de pago	Registro histórico de todas las compras realizadas.
gamesBox	Catálogo cacheado desde la API RAWG	Acceso offline al catálogo descargado previamente.

5.3 SharedPreferences

Clave (Key)	Propósito
last_customer_name	Recuerda el nombre del cliente para autocompletar el formulario.
last_customer_email	Email del cliente para autocompletar en la siguiente sesión.
last_customer_city	Ciudad del cliente para agilizar futuras compras.
preferred_payment	Último método de pago seleccionado (efectivo/tarjeta).
total_purchases_count	Contador de compras mostrado como estadística en el Home.

5.4 Animaciones

Animación 1 — Selección de Juego (AnimatedContainer)

Ubicación: Tarjetas de juego en add_game_screen.dart. Al tocar una tarjeta, AnimatedContainer cambia el color de fondo y el grosor del borde con Curves.easeInOut en 200ms. Los juegos seleccionados muestran borde cyan brillante.

Animación 2 — Compra Exitosa (ScaleTransition + AnimationController)

Ubicación: success_screen.dart. El ícono de verificación verde crece desde escala 0 hasta 1.0 en 600ms con Curves.elasticOut, creando un efecto de 'pop' satisfactorio al confirmar la compra.

Animación 3 — Estadísticas del Home (AnimatedOpacity)

Ubicación: home_screen.dart. Las cuatro tarjetas de estadísticas aparecen gradualmente con stagger de 150ms entre cada una, evitando que todos los elementos aparezcan simultáneamente al cargar la pantalla.

6. Dependencias del Proyecto (pubspec.yaml)

Paquete	Versión	Uso en el Proyecto
flutter	SDK	Framework principal multiplataforma.
hive	^2.2.3	Base de datos local NoSQL para favoritos e historial.
hive_flutter	^1.1.0	Integración de Hive con Flutter.
shared_preferences	^2.2.2	Preferencias simples del usuario.
http	^1.1.0	Peticiones HTTP para la API RAWG.
provider	^6.1.1	Gestión de estado global (carrito de compras).
flutter_launcher_icons	^0.13.1	Generación del ícono en todas las resoluciones.
build_runner	^2.4.7	Generación de código (adaptadores Hive TypeAdapter).
hive_generator	^2.0.1	TypeAdapter generado para los modelos Hive.

8. Retos Técnicos y Soluciones

Reto 1 — Gestión del Estado en la Tienda

Problema: Mantener sincronizado el estado de selección de múltiples juegos entre pantallas y al regresar desde la pantalla de pago.

Solución: Se utilizó onGenerateRoute en main.dart para pasar como argumentos la lista de juegos seleccionados (List<Map<String,dynamic>>), datos del cliente, totalItems y totalPrice, sin depender de estado global.

Reto 2 — Null Safety con Objetos Hive

Problema: Los objetos recuperados de Hive con el operador ?. generaban warnings de null-aware innecesarios al mapearlos de HiveObject a Game.

Solución: Se aplicó el operador ?? con valores por defecto en cada campo (fav?.name ?? '', fav?.rating ?? 0.0, etc.), garantizando null safety sin excepciones en tiempo de ejecución.

Reto 3 — Paginación Infinita en GamesListScreen

Problema: La API RAWG devuelve resultados paginados; cargar todo el catálogo en una sola petición agotaba los límites de la API y degradaba el rendimiento.

Solución: Se implementó paginación con _currentPage, _isLoadingMore y _hasMoreGames. Al detectar scroll hasta el final o al presionar el botón 'Cargar más', se incrementa _currentPage y se acumulan los resultados en _games.

Reto 4 — Diseño del Tema Oscuro Gaming

Problema: Implementar diseño gaming visualmente consistente sin que los múltiples colores de acento resultaran caóticos.

Solución: Se definió un ThemeData centralizado en main.dart con colores constantes (cyan para selección, rojo/verde/naranja para estadísticas) declarados en app_constants.dart y aplicados sistemáticamente en todos los widgets.

9. Conclusiones y Próximos Pasos

9.1 Reflexión sobre el Aprendizaje

El desarrollo de Game Vault permitió al equipo consolidar los conocimientos de Flutter: navegación con Named Routes y paso de argumentos, manejo de estado con StatefulWidget, consumo de API REST paginada con http, persistencia con Hive (null safety, TypeAdapters) y SharedPreferences, animaciones con ScaleTransition y AnimatedContainer, y un diseño gaming coherente con ThemeData centralizado.

9.2 Mejoras Futuras

- Autenticación de usuarios con Firebase Auth.
- Sistema de reseñas y calificaciones de juegos por parte de los usuarios.
- Integración de pasarela de pago real (Stripe o MercadoPago).
- Panel de administrador web para gestión del catálogo y estadísticas de ventas.
- Notificaciones push con Firebase Cloud Messaging para ofertas y nuevos títulos.
- Soporte multiplataforma para iOS.
- Modo claro (light theme) como alternativa al tema oscuro actual.